

G09 · 시스템 모델링 관점 — state · 비용 · 메모리계층 · scaling

이 권은 앞의 여덟 권에서 개념으로 쌓은 것 (메모리 = 읽으며 계속 학습되는 작은 모델, surprise, momentum, forgetting, optimizer memory, wake/sleep) 을 **시스템 엔지니어의 숫자로 옮긴다**. 목표는 하나다 — 이 계열 (Titans·Miras·Atlas·TNT·Nested Learning·Sleep) 을 배포했을 때 “decode 한 토큰에 무슨 일이 얼마나 일어나는가” 를 KV cache 와 나란히 놓고 **직접 모델링할 수 있게** 만드는 것. 수식은 보여주되 증명하지 않고, 모든 숫자는 비율 · 순서 · 교차점만 믿는다 (절대치는 미검증 하한임을 뒤에서 못박는다).

9.1 한 장의 그림: 무엇이 바뀌었나

transformer inference 를 아는 사람에게 이 계열을 한 문장으로 설명하면이렇다. **추론 중에 weight 가 변한다**. 오늘의 서빙 스택 전체가 딥고선 불변식 — “추론 중 weight 는 고정, 변하는 것은 KV cache 뿐” — 이 이계열에서 폐기된다. decode 가 토큰마다 자기 weight (정확히는 fast-weight state) 를 고쳐 쓰고, 배포에는 주기적 학습 job (sleep) 이 붙는다.

직관 오늘의 LLM 은 “다 배운 책” 이다 — 읽는 동안 책은 안 바뀌고, 지금까지 읽은 페이지 (KV cache) 를 옆에 쌓아 두고 참조만 한다. 이 계열의 모델은 “읽으며 여백에 계속 필기하는 책” 이다. 토큰 하나를 읽을 때마다 본문 (weight) 자체에 밑줄을 긋고 메모를 남긴다. 참조할 페이지 더미 (KV) 대신, 계속 고쳐지는 **작은 공책 하나**가 문맥을 흡수한다.

이 한 번의 뒤집힘이 이 권의 모든 결과를 만든다. state 의 성격 (append vs 고쳐쓰기), 비용의 병목 (용량 vs 대역폭), batching (공짜 vs 아님), 메모리계층 배치 (문맥 길이 vs 갱신 빈도), scaling 축 (문맥 vs 폭) — 전부가 이뒤집힘의 따름정리다.

시스템 모델링 관점 이 권을 관통하는 대비를 먼저 표로 박아 둔다. 이후 모든 절은 이 표의 한행씩을 푸는 것이다.

축	KV cache (오늘)	TTT state (이 계열)
상태 성격	append-once / read-many	매 토큰 read-modify-write(RMW)
write/read 비	~0.003% (거의 read 만)	100% (대칭)
공유 가능?	yes (prefix 재사용)	사실상 0 (내용이 매 토큰 바뀜)
크기 결정	문맥 길이 S 에 비례	모델 폭 d^2 에 고정 (문맥무관)
배치 병목	용량 (capacity) 이 먼저	대역폭 (bandwidth) 이 먼저
decode 병목	점점 memory-bound	항상 memory-bound

9.2 state가 곧 weights 다 — 아키텍처별 상태크기

가장 먼저 답할 질문은 “무엇을, 얼마나 들고 있어야 하나” 다. 이 계열에서 서빙 상태는 KV cache가 아니라 **fast-weight 행렬 자체**다. 그 크기는 아키텍처 class가 정한다.

수식으로는 layer 하나당 상태 크기가 $m d^2$ 개의 숫자이고, 이걸 전체로 곱하면 $m d^2 L_{\text{layer}}$ 다.

기호 풀이 d = hidden 차원 (모델 폭, anchor에서 2048). $d^2 = d \times d$ 행렬 하나의 원소 수. m = **state multiplier**, 상태가 d^2 의 몇 배인가 (아키텍처가 정하는 배수). L_{layer} = layer 수 (문맥 길이가 아니라 층 수, anchor 24). s = 숫자 하나의 byte 수 (bf16 이면 $s=2$). 곱 $m d^2 L_{\text{layer}} s$ = 모델 전체서빙 상태의 byte 크기.

이 식은 “서빙 상태 크기 = 폭의 제곱 $\times$ 층수 $\times$ 배수” 라는 뜻이다. 결정적으로 **문맥 길이 S 가 이 식에 없다**. KV cache는 문맥이 길수록 커지지만, 이 상태는 문맥과 무관하게 폭으로만 커진다.

m 사다리가 아키텍처를 순서 짓는다. 앞 권들에서 본 각 설계가 여기서 하나의 숫자가 된다.

표 9-1 — 아키텍처 class 별 상태 크기 (anchor 폭 $d = 2048$, $L_{\text{layer}} = 24$, bf16). 절대 GB는 표준 shape 가정에서 나온 값이고, **load-bearing** 한 것은 **class 간 순서와 “전부 memory-bound”** 라는 판정뿐이다.

아키텍처 class	앞 권	m	state/layer	토큰당 RMW 트래픽	bound
matrix / linear memory (+momentum)	G02·G03	≈ 2	≈ 17 MB	≈ 0.8 GB	memory
deep memory (2-layer MLP)	G03 Titans	≈ 8	≈ 67 MB	≈ 3.2 GB	memory
+ momentum (Titans-LMM, anchor)	G03·G04	≈ 16	134 MB	6.44 GB	memory
Hope (6-memory 블록)	G07	상주 합 최대	상주 최대	fast-level 지배	memory

직관 메모리를 “깊게”(matrix $\rightarrow$ deep-MLP) 만들거나 momentum을 없으면 capacity가 오르는 대신 상태가 무거워진다. deep-MLP는 weight 행렬 두개 (W_1, W_2)를 들고, momentum은 그 각각의 “관성 버퍼” S_t 를 하나 더 얹는다. Hope는 한 블록 안에 여섯 개의 작은 메모리 (main + 다섯 projection)를 품어 상주 상태가 최상단

이다.

기호 풀이 S_t = momentum buffer(최근 surprise 방향의 관성을 담는 별도 행렬, G04). W_1, W_2 = deep-memory MLP 의 두 weight 행렬. “6-memory 블록” = Nested Learning 이 k, v, q, η, α 투영까지 test-time 에 갱신되는메모리로 바꾼 self-modifying 구조 (G07).

핵심 상태 크기의 배수 m 은 아키텍처가 정하고 (matrix 1 $\rightarrow$ +momentum 2 $\rightarrow$ deep-MLP 8 $\rightarrow$ +momentum 16 $\rightarrow$ poly ~ 24), 그 순서가 그대로**서빙 비용의 순서**다. 무거운 메모리는 “더 빠른” 것이 아니라 “매 토큰 더 많은 byte 를 옮겨야 하는” 것이다.

9.3 decode 한 스텝: 왜 memory-bound 인가

이제 핵심 질문. decode 토큰 하나를 처리할 때 하드웨어가 실제로 무슨 일을하나?

토큰마다 fast-weight state 를 **읽고 (read), 갱신하고 (modify), 되쓴다 (write)** — 이것이 RMW 다. 읽기가 $m d^2$ byte, 쓰기가 $m d^2$ byte, 그래서 layer 마다 상태 크기의 두 배를 옮기고 L_{layer} 층에 걸쳐 누적된다. anchor 에서 이 값은 **6.44 GB/token** 이다.

비유 매 글자를 읽을 때마다 공책 전체를 처음부터 끝까지 다시 베껴 쓰는 것. 글자 하나 (토큰) 를 받으면 공책 (state) 전부를 읽어서, 조금 고친 뒤, 전부다시 쓴다. 공책이 클수록 (폭이 넓을수록) 글자 하나에 드는 베껴쓰기 노동이 커진다. 정작 “생각하는”(연산하는) 시간은 그 베껴쓰기에 완전히 묻힌다.

그 “생각” 이 얼마나 묻히는지는 arithmetic intensity(AI) 로 잰다. AI 는 옮긴 byte 당 몇 번 계산하는가다.

기호 풀이 AI(arithmetic intensity) = FLOP $\div$ byte, 즉 데이터를 1 byte 옮길 때 산술연산을 몇 번 하는가. ridge = 그 하드웨어에서 “연산이 대역폭을 꽉채우는” 분기점 (H100 twin 기준 ≈ 295 FLOP/byte). AI < ridge 이면 대역폭이 병목 (memory-bound), AI > ridge 이면연산이 병목 (compute-bound).

anchor decode 의 AI 는 약 **0.59 FLOP/byte** 로, ridge 295 보다 두자릿수 아래다. 즉 이 스텝은 **결정적으로 memory-bound** 이고, 상태트래픽이 실제 GEMV 연산을 약 **394 $\times$ 압도한다**. 한 문장으로: **decode step 의 비용은 곧 state 트래픽이다**. 연산은 공짜에 가깝고, 시간의 거의 전부가 6.44 GB 를 읽고 되쓰는 데 든다.

이 성질은 아키텍처를 무겁게 해도 안 바뀐다. deep-MLP 로 가면 연산도늘지만 트래픽도 함께 $m d^2$ 로 늘어 AI 는 상수에 묶인다. Atlas 의 Newton – Schulz 반복 (NS₅) 처럼 그 자체로는 compute-heavy 한연산조차, 무거운 상태를 왕복하는 트래픽 시간에 가려 step 전체는 memory-bound 로 남는다.

시스템 모델링 관점 decode 한 스텝을 모델링하는 최소 공식은 이것이다. - 옮기는 byte = $2 m d^2 L_{\text{layer}} s$ (state 한 번 read + 한 번 write). anchor 에서 6.44 GB. - **latency** 하

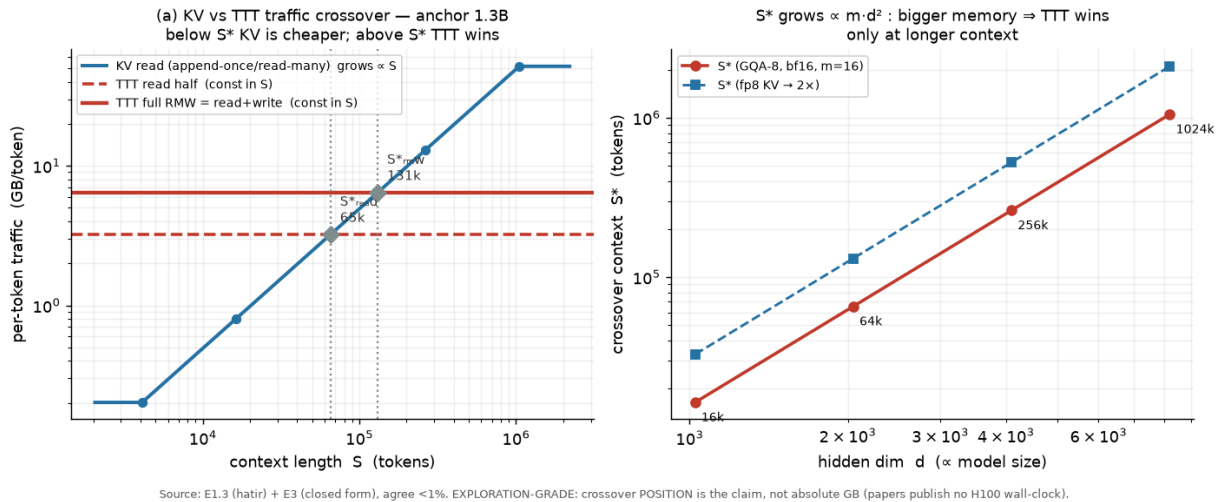


그림 9-1 — KV cache 읽기 트래픽 (문맥에 비례 증가) 과 TTT state RMW 트래픽 (문맥 무관 일정) 이 만나는 crossover 문맥 길이, 그리고 그 폭 scaling. 출처: 저자 자체 실험 그림 exp-a

한 = (옮기는 byte) ÷ (HBM 대역폭). 연산 시간은 무시해도 된다 ($394 \times$ 지배). - **data dependency** = 다음 토큰의 state 는 이번 토큰의 state 에 의존 (recurrence). 그래서 토큰 축은 순차적, 병렬화 불가. - **KV 와 대비** = KV 는 append 후 read-many(쓰기 거의 0), TTT 는 매번 대칭 RMW(쓰기 100%). 같은 “memory-bound” 라도 종류가 다르다.

실전 규칙: 이 계열 decode 를 sizing 할 때 FLOP roofline 은 잊어라. “**매 토큰 state 전체 $\times 2$ 를 대역폭으로 나눈다**” 가 첫 번째 근사이자 대개 마지막 근사다.

주의 anchor 의 절대 하한 (1.92 ms/token, 46.5 mJ/token) 은 이 권의 어떤 주장의 근거도 **아니다**. 원 논문 여섯 편이 decode wall-clock 을 하나도 공개하지 않았기 때문에, 이 절대치는 외부 검증 불가능한 roofline 하한이다. 믿을 것은 “ $394 \times$ state-지배”, “AI $\ll$ ridge”, “class-무관 memory-bound” 라는 구조뿐이다.

9.4 KV cache 와의 질적 차이, 그리고 crossover

두 부하는 roofline 의 같은 축 (대역폭) 에 걸리지만 성격이 정반대다.

- **KV cache**: 토큰마다 몇 KB 를 덧붙이고 (append), 그 뒤로 여러 번 읽기만 한다. 쓰기는 read 의 약 0.003%. prefix 가 같으면 여러 요청이 공유할 수 있다.
- **TTT state**: 토큰마다 상태 전체를 읽고 되쓴다. 쓰기 = 읽기 (100% 대칭). 내용이 매 토큰 바뀌므로 요청 간 공유가 사실상 0.

여기서 시스템 모델링의 핵심 교차점이 나온다. KV 의 읽기 트래픽은 문맥이 길수록 커지고 ($\text{read} \propto S$), TTT 의 RMW 트래픽은 문맥과 무관하게 일정하다 (크기 $m d^2 L_{\text{layer}}$ 고정). 두 곡선이 반드시 어딘가에서 만난다 — 그 문맥 길이가 **crossover** S^* 다.

기호 풀이 S = 문맥 길이 (토큰 수). S^* = crossover 문맥 길이, 이아래에선 KV 가 싸고 위에선 TTT 가 싸다. anchor(1.3B) 에서 read-crossover $\approx 65k$ 토큰, 전체 RMW-crossover $\approx 131k$ 토큰 (약 2 배). S^* 는 모델 폭에 따라 $m d^2$ 로 이동한다.

직관 KV 는 “읽을수록 쌓이는 노트 더미” 라 문맥이 길면 무거워진다. TTT 는 “고정 크기 공책” 이라 문맥이 아무리 길어도 크기가 그대로다. 그래서 **짧은 문맥에선 노트 더미 (KV) 가 가볍고, 긴 문맥에선 고정공책 (TTT) 이 가볍다**. 두 방식은 문맥 축에서 서로의 영역을 나눠 가진다.

시스템 모델링 관점 이 crossover 가 배포 배치 규칙을 준다. - 짧은 문맥 · 다중 테넌트 (챗봇류, 문맥 $\ll 65k$) $\rightarrow$ 오늘의 KV cache 가 여전히 싼 메모리 시스템. - 긴 문맥 · 소수세션 (문서 · 에이전트류, 문맥 $\gg 131k$) $\rightarrow$ TTT state 가 싸다 (RMW latency 가 문맥무관하게 일정). - 완성형은 아마 **hybrid** 다: 짧은-문맥 성분은 sliding-window attention (공유 · batchable) 으로, 긴-문맥 성분만 TTT state 로. crossover 의 양쪽을 한 모델이 나눠 갖는다.

S^* 는 폭에 따라 **16k $\rightarrow$ 1.05M 토큰**으로 이동한다 (작은모델일수록 일찍, 큰 모델일수록 늦게 뒤집힌다). fp8 로 KV 를 양자화하면 S^* 가 2 배로 밀린다.

9.5 batching: 묶을 순 있지만 상각은없다

서빙은 여러 요청을 동시에 흘린다. 오늘의 dense LLM 에서 batching 은 거의공짜다 — 모든 요청이 **같은 weight** 를 공유하므로, weight 를 한 번읽어 B 개 요청에 재사용한다 (shared-weight GEMM). 그래서 batch 가 커질수록 AI 가 오르고 대역폭이 상각된다.

이 계열은 이 free lunch 가 깨진다. 요청마다 fast-weight W_b 가 **다르기** 때문이다.

기호 풀이 B = batch 크기 (동시 요청 수). W_b = 요청 b 의 fast-weight 상태 (요청마다 다름). shared-weight GEMM = 모든 요청이 같은 weight 를 쓰는행렬곱 (상각됨). grouped-GEMM = 요청마다 다른 weight 를 각자 도는 묶음행렬곱 (상각 안 됨).

직관 오늘의 batching 은 “학생 30 명에게 같은 교과서 한 권을 읽어 주기” — 책은 한 번 펴면 30 명이 공유한다. 이 계열의 batching 은 “학생 30 명이 각자다른 필기 공책을 들고 있어, 30 권을 전부 따로 읽고 따로 고쳐 써야 하기” — 묶어서 한 번에 처리 (dispatch) 는 되지만, 공책마다 노동이 따로 든다. 30 명이라고 노동이 상각되지 않는다.

수식으로는 이렇다. 요청별 state 를 batch 축으로 쌓으면 read 는 $Y[b] = X[b] M[b]^T$ ($b = 1, \dots, B$) 꼴의 **묶음행렬곱 (BMM)** 이 된다. 문제는 각 W_b 가 그 요청에만 쓰여 한 번 읽고한 번 써진다는 것 — FLOP 도 트래픽도 함께 B 에 비례하므로 AI 가 **batch B 와 무관**하다.

기호 풀이 $M[b]$ = 요청 b 의 상태 행렬 ($[B, d, d]$ 의 한 장). $X[b]$ = 요청 b 의 입력. BMM(batched matmul) = 요청별로 독립인 작은 행렬곱을 batch 차원에 쌓은 연산.

이것이 **never-average** 원칙이다 — 요청 b 의 momentary surprise를 다른 요청과 평균하면 한 테넌트의 문맥이 다른 테넌트로 샌다 (정보 누출). 그래서 inner-loop update는 요청 축으로 **map** 이지 reduction 이 아니다. batch는 형성되되 상각되지 않는다.

시스템 모델링 관점 batching의 실전 결론. - **가능성 (feasibility)**: batch는 grouped-GEMM으로 형성된다 — dispatch 골격은 MoE와 닮았다. - **경제학 (economics)**: shared-weight 상각은 **없다** — AI가 B 와 무관해 대역폭이 배치를 조기에 달는다. - $B_{\max}$ **벽**: 10 ms/token 목표에서 대역폭 상한 배치는 **340M=20.8, 1.3B=5.2, 7B=0.97 sequence**. 7B에선 batch-1조차 목표 latency 안에 상태를 한 번 통과시키기 어렵다. - **KV와 대비**: KV는 용량 (capacity) 이먼저 막히지만 TTT는 **대역폭**이 먼저 막힌다 (bandwidth wall).

모델링 실전: KV서빙은 “HBM에 몇 개 세션이 들어가나”로 batch를 잡지만, 이 계열은 “ $2 B m d^2 L_{\text{layer}}$ 가 대역폭 예산 $BW \cdot t_{\text{target}}$ 을 언제 채우나”로 $B_{\max}$ 를 잡는다. 완화책은 fp8 상태 (트래픽 절반), per-layer streaming + double-buffering, prefill/decode 분리, hybrid(attention 절반에서 batch 회복) 다.

한계 $B_{\max}$ 절대값 (0.97 등)은 roofline 하한이자 weights/activations/KV 공존을 뺀 upper bound라 A100 실측 대기다. load-bearing한 것은 “7B/10ms에서 batch가 한 자릿수로 몰린다”는 **class와 방향**이지 “0.97”이라는 숫자가 아니다.

9.6 decode에 들어온 backward pass — 새 serving primitive

RMW의 byte 수는 무엇이 옮겨지는지 말하지만, 무엇이 계산되는지는 말하지 않는다. 그리고 계산되는 것 안에 이 계열이 서빙 시스템에 던진 진짜새로움이 있다.

decode의 forward는 익숙하다 — state를 한 번 읽는다 ($y_t = \mathcal{M}(q_t; W_t)$). 그런데 그 다음 state를 갱신하려면 inner loss의 gradient가 필요하고, 그것을 얻으려면 memory MLP를 거슬러 오르는 **backward pass**가 있어야 한다.

기호 풀이 $\mathcal{M}(q_t; W_t)$ = 상태 W_t 로 query q_t 를 읽은 결과. q_t, k_t, v_t = 토큰 t 의 query·key·value. $g_t^{\text{in}} = \nabla_W \ell(W_{t-1}; k_t, v_t)$ = inner loss의 gradient(state를 어느 방향으로 고칠지, “얼마나 틀렸나 = surprise”의 방향). η_t = inner learning rate(이번 놀람을 얼마나 반영할지). α_t = retention gate(0~1, 조금씩 지우기 = forgetting). 갱신: $S_t = \beta_t S_{t-1} - \eta_t g_t^{\text{in}}, W_t = \alpha_t W_{t-1} + S_t$.

이 식은 “surprise 방향 (g_t^{in})을 momentum(S_t)으로 누적해, 조금 지운 (α) 옛 상태 위에 얹는다”는 뜻이다 — G01~G04에서 개념으로 본 그대로다. 시스템 관점에서 새로운 것은 이 계산이 **훈련의 backward pass와 같은 연산**이라는 점이다.

직관 오늘의 inference kernel(FlashAttention decode, paged attention)은 정의상 **forward-only**다 — “추론 중 weight 불변”이라는 전제 위에 세워졌기 때문. 이 계열은 그 전제를 깨서, “훈련에만 있던” forward+backward의 합성이 토큰마다 한 번씩 서빙 loop 안에서 돈다. byte 회계는 결과일 뿐, **진짜 새 primitive는 backward**

pass가 decode의 critical path 위로 올라왔다는 사실 자체다.

시스템 모델링 관점 이 primitive가 kernel에 요구하는 것 셋. 1. **fused forward+backward decode kernel** — read·backward·momentum·retention·write를 한 번의 state streaming 안에 융합해야 한다. 안 하면 kernel마다 state를 따로 왕복해 트래픽이 4~6 배로 부른다. 트래픽이 이미 비용의 전부이므로, fusion은 “몇 % 튜닝”이 아니라 하한에 얹느냐 그 4~6 배 위에 얹느냐의 문제다. 이런 kernel은 아직 존재하지 않는다. 2. **activation residency**는 오히려 쉽다 — 훈련 backward가 어려운건 sequence 전체 activation을 쌓아야 해서인데 ($O(L \cdot md)$), decode backward는 **한 토큰**의 backward라 유지할 activation이 $O(md)$ 뿐. 문맥 축 activation 스택이 없다. 훈련 backward의 가장 악명높은 비용이 decode에선 구조적으로 사라진다. 3. **checkpoint/rollback**이 새 상태 이벤트 — speculative decoding의 rollback이 KV에선 포인터 되감기(공짜)지만, 여기서 fast-weight W_t + momentum S_t 까지의 **snapshot/restore**다. 되돌리는 게 optimizer 궤적이라 부정확하면 numerics가 표류한다.

9.7 메모리 계층 배치 — fit → spill 절벽과 cadence → tier

decode가 memory-bound이고 그 병목이 whole-state RMW라면, 남은 설계자유도는 **그 state를 어디에 두는가**다. 여기서 KV와의 결정적 차이가 유리하게 작동한다 — TTT state는 문맥 무관 고정 크기라 **on-chip 상주 여부가 설계 가능한 knob**이 된다 (문맥에 따라 자라상주 계획을 못 세우는 KV와 다르다).

첫째, 상주의 물리적 한계. on-die L2(50 MB 급)는 anchor에서 한 sequence의 whole-model state조차 못 담는다. 따라서 상주는 whole-model pin이 아니라 **per-layer/streamed**여야 한다.

state가 268 MB scratchpad에 fit하는 동안은 scratchpad가 HBM3 대비 $6.8 \times \text{energy} / 14.9 \times \text{time}$ 이득을 낸다. 그러나 폭이 커지면 ($d = 4096, 7B$) 한 layer state가 537 MB로 커져 **spill**한다 — residency crossover는 $d^* \approx 2896$.

둘째, 그 spill의 대가. 상주 경계를 넘는 순간 대역폭이 부드럽게나빠지는 게 아니라 **절벽처럼 꺾인다**.

in-cache RMW는 ~265 GB/s로 돌지만, 용량을 넘겨 DRAM으로 spill하면 ~68 GB/s로 약 $3.9 \times$ 붕괴한다. 게다가 RMW는 element당 read+write로 $2 \times \text{byte}$ 를 옮긴다 — append-once KV가 피하는 **write-back 세(稅)**를 직접 낸 값이다.

직관 상태를 캐시 안에 붙잡아 두면 빠르지만, 조금이라도 넘치면 속도가 절벽에서 떨어진다 ($3.9 \times$). 그리고 이 부하는 매번 “읽고 되쓰기”라, 같은대역폭이라도 실질 처리량이 read-only의 절반이다 (write-back 세).

셋째, 어느 tier에 두나 = 갱신 빈도가 정한다. Nested Learning·Sleep이 각 메모리 level에 준 update cadence가 그대로 tier 배치명세가 된다.

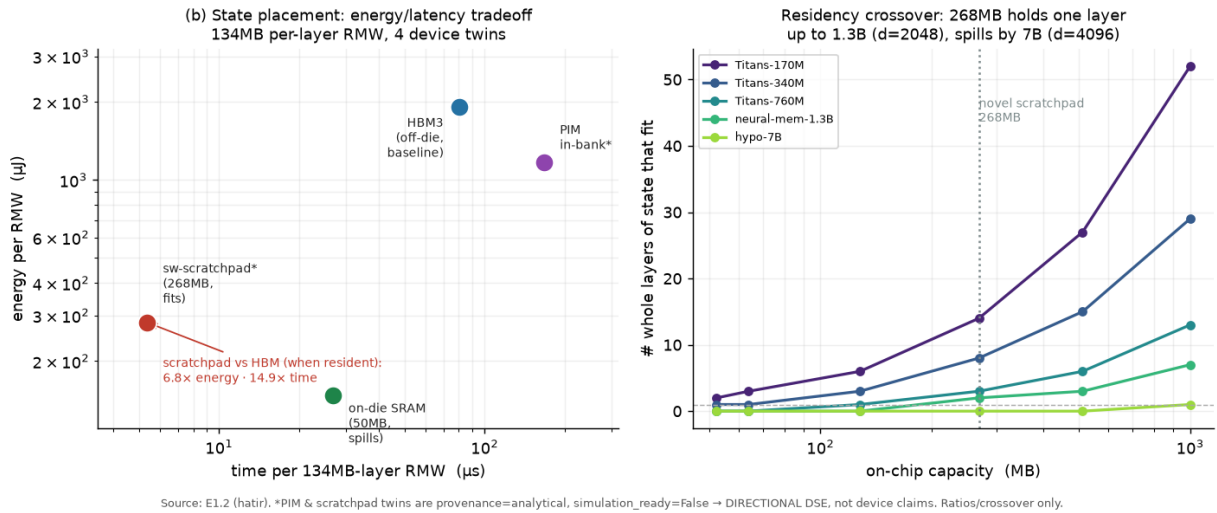


그림 9-2 — per-layer RMW state 를 device 별로 배치했을 때의 energy/latency 와 residency crossover: state 가 fit 하는 쪽에서는 scratchpad 가 HBM 을 이기고, 폭이 커지면 spill 한다. 출처: 저자 자체 실험그림 exp-b

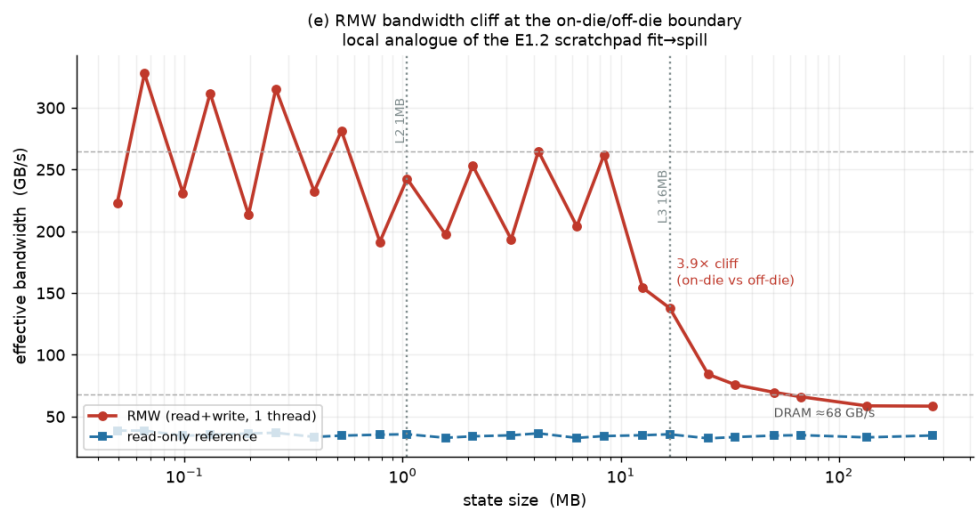
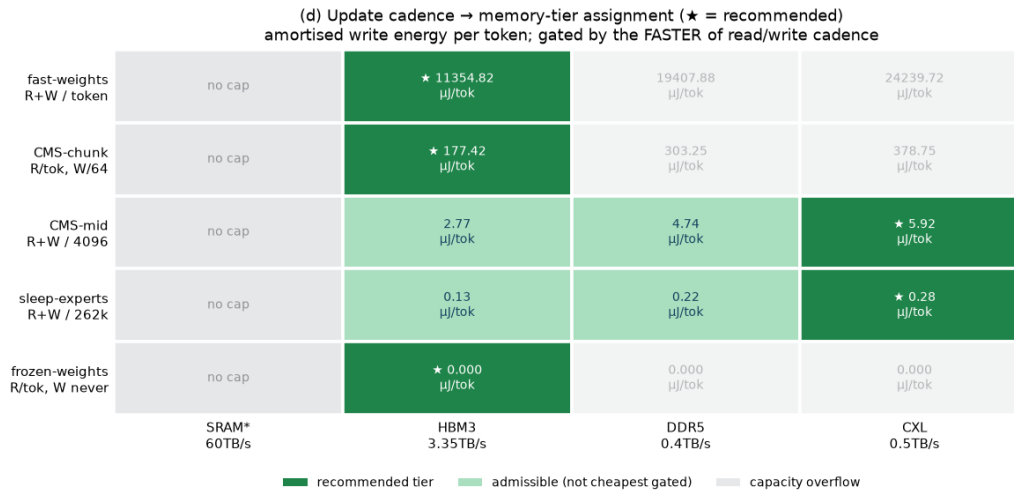


그림 9-3 — on-die/off-die 경계에서 RMW 대역폭 cliff: in-cache RMW 가 capacity 경계를 넘으면 유효 대역폭이 불연속으로 붕괴한다. 출처: 저자 자체 실험 그림 exp-e



Source: E1.4 (hatir). *SRAM tier = novel scratchpad twin (simulation_ready=False, directional). Sub-per-token cadence lets a write-heavy RMW state legally drop to DDR/CXL without lengthening the critical path.

그림 9-4 — update cadence 에서 memory-tier 로의 번역: 상주 사본은 read/write 중 빠른 쪽 cadence 로 tier 가 정해진다. 출처: 저자 자체 실험그림 exp-d

기호 풀이 cadence = 갱신/접근 주기 (몇 토큰마다 read/write 하나). tier = 메모리계층 (HBM3=빠름/뜨거움, CXL·DDR=느림/차가움). gating rule = “상주 사본은 read/write 중 빠른 쪽 cadence 로 tier 가 정해진다”.

표 9-2 — update cadence → memory-tier 배정 (anchor).

memory level	cadence (read/write)	tier	근거
fast-weights	토큰 / 토큰	HBM3	가장 뜨거운 admissible tier
CMS-chunk	토큰 / 64 토큰	HBM3	read 가 매 토큰이라 pin
CMS-mid	4096 / 4096 토큰	CXL	down-sampled → 느린 tier 허용
sleep-experts	262144 / 262144 토큰	CXL	offline cadence

시스템 모델링 관점 이걸 오늘의 서빙 엔지니어에게 익숙한 문법의 재서술이다 — paged KV cache 가 hot page 를 HBM 에, cold page 를 host/CXL 로 내리는 것과 같다. 다른 점은 “무엇이 hot 인가”를 문맥 길이가 아니라 **update frequency**가 정한다는 것. 매 토큰 읽히는 fast-weight 는 HBM 에 pin 되고, 4096 토큰마다 접근되는 CMS-mid 는 CXL 로 합법적으로 내려간다 (드문 접근이 트래픽을 critical path 아래로 amortize 한다). 단 조건이 하나 — write 가 4096 주기라도 **read 가 매 토큰이면** gating rule 이 그것을 HBM 에 pin 한다. CXL 강등은 그 block 이 forward 에서 genuinely 건너뛰어질 때만 성립한다.

한계 scratchpad 의 $6.8 \times / 14.9 \times, 7.4 \times$ energy 이득은 simulation_ready=False 인 **directional DSE** 다 — 배치의 방향 (per-layer streamed 면 on-chip 이득이 실재) 만 load-bearing 이고, 배율은 silicon 전까지 device 주장이 아니다. cliff 의 GB/s 절대값도 host 좌표라 H100 으로 이전 안 된다 — 이전되는 것은 cliff 의 존재와 $3.9 \times$ 모양뿐이다.

9.8 scaling: sizing 숫자가 둘에서셋으로

transformer 를 서빙하는 엔지니어는 두 숫자로 fleet 을 계획한다 —params(weight 상 주분, 고정) 와 문맥 길이 (KV cache, 세션마다 자람). 이제열은 세 번째 숫자를 강제한다.

state-bytes 축 (cost 쪽). decode cost 의 scaling 변수는 tokens D 가 아니라 $B_{\text{state}} = m d^2 L_{\text{layer}}$ s 다. 이 상태가 매 토큰 RMW 되므로, 트래픽이 폭에 따라 자란다.

표 9-3 — 모델 폭에 따른 decode RMW 트래픽 $\cdot \text{crossover} \cdot B_{\text{max}}$ scaling(anchor $m = 16$; 문맥 S 에 무관).

규모	RMW GB/token	S^* read-crossover	B_{max} @10ms
Titans-170M	0.45	—	—
Titans-340M	1.61	16,384	20.8
Titans-760M	3.62	36,864	9.24
neural-mem-1.3B	6.44	65,536	5.2
hypo-7B	34.36	262,144	0.97
hypo-70B	343.6	1,048,576	0.1

한 줄로: **decode 트래픽은 폭에 따라 $0.45 \rightarrow 6.44 \rightarrow 343$ GB/token 으로 자란다.** 70B 에서 토큰당 343 GB 는 단일 80 GB HBM 에 안 담겨 sharding 이 전제되고, 집계 HBM3 대역폭으로도 state 이동만으로 토큰당 100 ms 규모 하한이 나온다.

capacity 축 (quality 쪽, 조건부). 두 번째 빠진 축은 memory capacity 다 — 정확히 저장 가능한 (key,value) 쌍의 최대 수로, class 별로 다르다 (matrix $O(d_k)$, poly $O(d_k^p)$, attention 무한, G05·G02).

기호 풀이 capacity = 메모리가 exact 하게 저장할 수 있는 linearly-independent (key,value) 쌍 최대 수 (아키텍처 class 순서를 정함). state-bytes = 상태의 물리적 byte 크기 (B_{state}). ppl = perplexity(언어모델품질, 낮을수록 좋음). retention = 긴 문맥에서 정보를 유지하는 길이 (BABILong 등). ρ = Spearman rank correlation(순위 상관).

capacity 가 언제 값을 하는지가 이 그림의 요점이고, 답은 **target 에 따라 갈린다**. - **ppl 축에서는 capacity 가 state-bytes 위로 아무것도 안 더한다** (둘이 rank-동치, $\rho = -0.949$). attention 을 넣으면 오히려 부호가 뒤집혀 ($\rho = +0.094$) **오도한다** —capacity 가 무한인 attention 이 ppl 은 오히려 나쁠 수 있어서다 (ppl 은 압축을 보상한다). - **long-context retention 축에서만 capacity 가 state-bytes 를 이긴다** ($\rho = 1.0$ vs 0.333). 결정적 증거: Titans $\rightarrow$ Atlas 에서 state-bytes 는 $1.5\times$ 만 늘었는데 retention 길이는 $\sim 6.7\times$ 된다 (poly feature 가 capacity class 를 하나 올려서).

주의 우하 panel 의 U-curve 는 scaling law 가 **아니다** — train/serve chunk 해상도 mismatch 다 (550M 을 $C=64$ 로 훈련하고 $C=8$ 로 서빙하면 ppl 13.78 $\rightarrow$ 36.45 로 무너짐, G06 TNT). 최소가 스케일이 아니라 train chunk 에 걸리는 게 그 증거다. scaling 표에 이 점을 섞으면 안 된다.

E4 — scaling-law digitization + fits (dossier A3): capacity is a long-context axis, not a perplexity axis

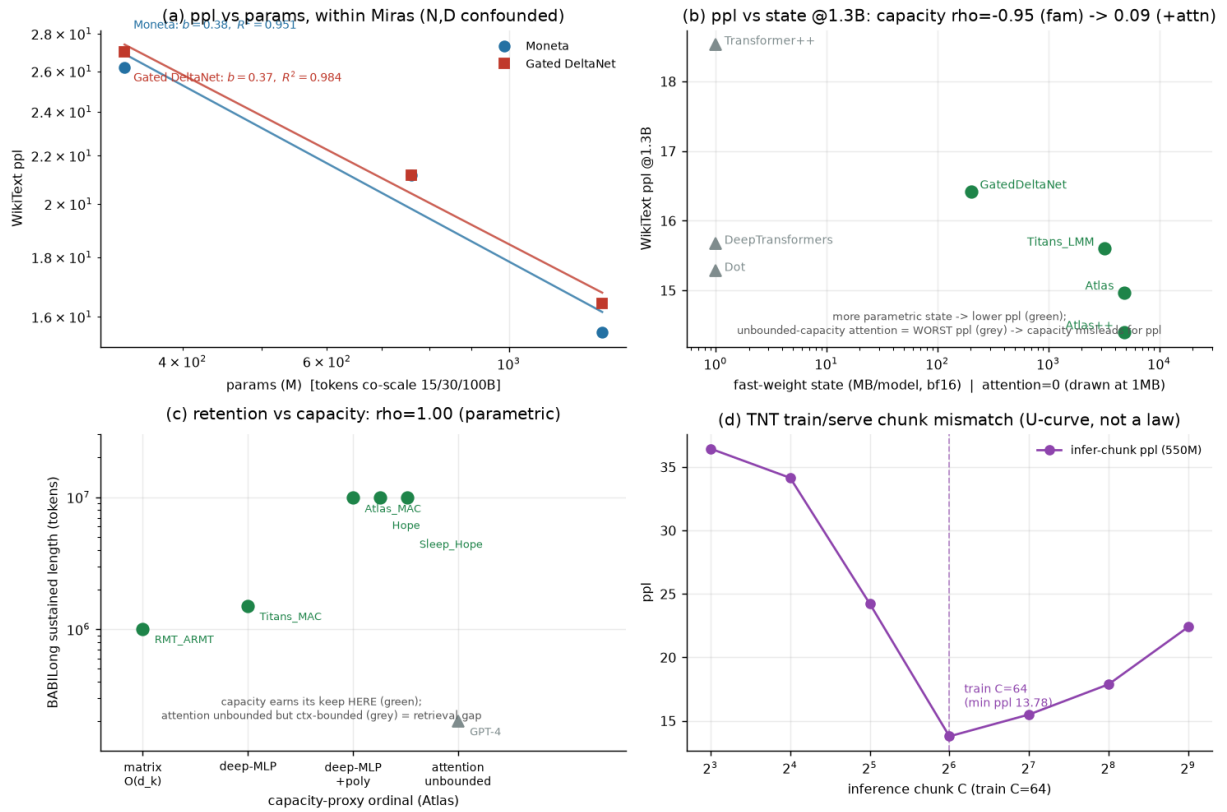


그림 9-5 — E4 scaling fit(2×2): (좌상) within-line ppl-vs-params fit 과 N,D confound, (우상) 고정 1.3B 에서 state-bytes·capacity 의 ppl rank test, (좌하) capacity 가 값을 하는 유일한축 BABILong retention 과 Titans → Atlas 도약, (우하) train/serve chunk mismatch 의 U-curve.
 출처: 저자 자체 실험 그림 exp-f

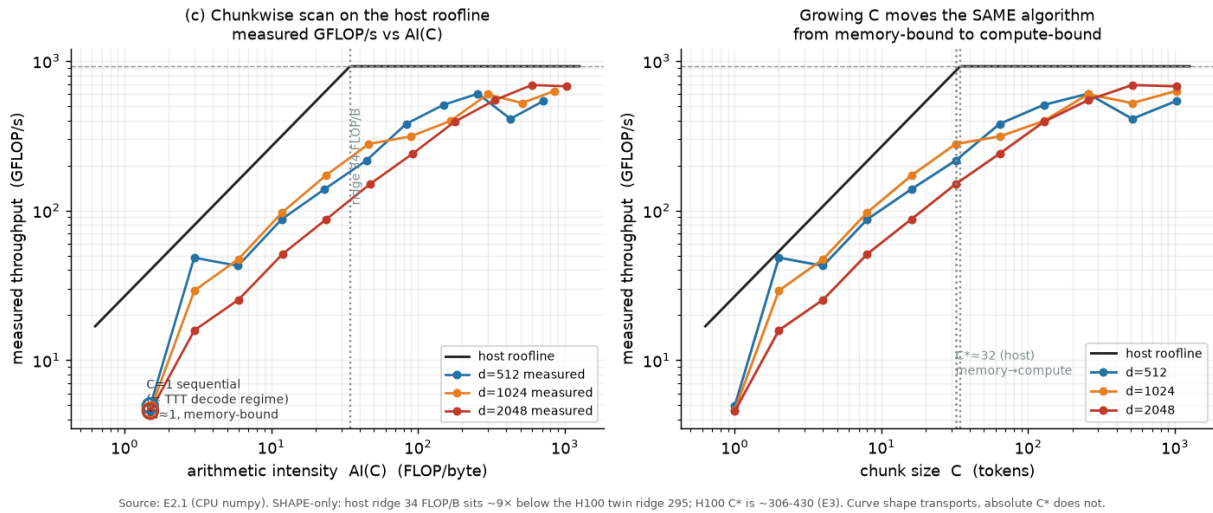


그림 9-6 — chunk 크기 C 에 따른 arithmetic intensity 의 roofline 상승: $C=1$ (decode 영역) 의 memory-bound 평원에서 C^* 의 compute-bound 영역으로. 출처: 저자 자체 실험 그림 exp-c

시스템 모델링 관점 fleet sizing 의 산수가 바뀐다. - 두 숫자 → 세 숫자: params(고정) + 문맥 길이 (KV, 세션마다 자람) + $B_{state} \propto m d^2 L_{layer}$ (per-session, 문맥 무관, 매 토큰 RMW). 세 번째는 KV 자리를 대신하되 성격이 정반대 — 폭으로 자라고, capacity-bound 가 아니라 bandwidth-bound 이며, on-chip 상주가 설계 knob. - **어느 축을 어느 목표에:** class 를 고를 때 ppl 만 보면 capacity 사다리는 잉여 (state-bytes 가 이미 rank 를 정함). 목표가 **long-context 유지** 라면 capacity-class 가 state-bytes 보다 나은 예측자 — 같은 GB/layer 라도 poly feature 로 class 를 올리면 retention 이 byte 증가분보다 크게 오른다.

9.9 chunk knob: 같은 알고리즘이 compute-bound 로 넘어가는 곳

지금까지는 decode($C=1$) 만 봤다. 하지만 **같은 알고리즘** 이 prefill/training 에선 compute-bound 가 된다. 그 스위치가 chunk 크기 C 다.

기호 풀이 C = chunk 크기 (한 번에 병렬 처리하는 토큰 수). $C=1$ = per-token RMW(decode 영역). C^* = memory ↔ compute 교차 chunk 크기. chunk 안 토큰들은 같은 chunk-시작 state 에서 gradient 를 평가하므로 (stale-snapshot 근사, G02), C 가 클수록 state 재사용이 늘어 AI 가 오른다.

이 그림이 pair thesis 를 한 장면으로 압축한다: **같은 knob C 가 decode 를 memory-bound 로, prefill/training 을 compute-bound 로 만든다** — 한 알고리즘, 두 영역. $C=1$ 은 host roofline 의 약 12%에 머무는 memory-bound 평원이고, C 를 키우면 C^* 를 넘어 compute-bound 로 오른다.

직관 chunk 를 키운다 = 공책 한 번 읽어 여러 글자를 한꺼번에 처리하기. 그러면 “베껴쓰기 노동” 이 여러 글자에 상각돼 연산 비중이 오른다. 그래서 decode(글자 하나씩)

는 대역폭 병목, prefill(글자 뭉치) 은 연산 병목이된다.

여기서 MFU(연산 활용률)가 왜 5~10%로 “새는지”가 보인다. 품질이 최적인 C 는 작고, 작은 C 는 memory-bound 평원 (roofline의 12%) 위에 있다. 즉 MFU는 kernel 결합으로 “새는” 게 아니라 품질을 위해 지불되는 것이다. TNT(G06)가 two-stage 훈련으로 chunk-1을 품질 최적점으로 옮긴 것은 이 작동점 자체를 옮겨 누수원인을 없앤 시도다.

한계 C^* 의 절대값은 ridge에 의존한다 — host CPU에서 $C^* \approx 32$ 지만 H100에선 306~430이다. 이전되는것은 “낮은 C 는 memory-bound, 높은 C 는 compute-bound”라는 곡선의 모양뿐, 절대값이 아니다.

9.10 하드웨어 제안은쌍 (pair) 이다

이 계열의 배포는 질적으로 다른 두 부하로 갈라지고, 각각의 최적 하드웨어전략이 다르다. 흥미로운 제안은 둘 중 하나가 아니라 쌍이다 — 한쪽만 옹호하면 부하의 절반을 무시하는 것.

memory-centric 절반 (decode). - **RMW-bandwidth decode 소자** — 높은 RMW 대역폭 + per-tenant state residency + write-heavy elementwise epilogue(decay·renorm·AXPY)를 위한 near-memory update engine. 근거: decode가 measured memory-bound whole-state RMW라는 사실 (부과가 아니라 논문들의 cost 회계에서 읽어 낸 것). - **frequency-tiered placement** — update cadence를 tier admissibility 규칙으로 소비 (§ 9.7). “NL/Sleep의 정의가 이미 memory-hierarchy 사양”이라는, 이 스터디에서 가장 신선한 memory-architecture 주장.

accelerator 절반 (prefill/training + decode의 연산부). - **fused chunk kernel** — momentum·retention 포함 deep-memory chunk를 하나로 융합. 이 계열이 아직 기다리는 “FlashAttention-moment”의 후보. (현재 flash-linear-attention에 Titans는 naive PyTorch 참조만 있고 진짜 fused kernel이 없다는 게 생태계 측 물증.) - **grouped-GEMM decode engine** — 요청별 unshared weight를 batch 축으로 묶어 kernel-launch·occupancy 파편화를 걷는다. AI 자체는 못 올리지만 (§ 9.5) 포획 손실을 줄인다. - **backward-capable serving kernel** — decode 경로에 inner-loop backward를 일급 연산으로 (§ 9.6).

시스템 모델링 관점 이 쌍이 한 decode step을 어떻게 나눠 맡는지가 모델링의 최종 그림이다. **state RMW 트래픽 하한**은 memory-centric 소자(D)가 흡수하고, **backward 연산과 batch 재조직**은 accelerator kernel($G \cdot H$)이 맡으며, **prefill의 compute-bound chunk**는 fused kernel(F)이 처리한다. G 와 D 는 경쟁이 아니라 같은 step의 분업이다 — G 는 batch를 형성하고, D 는 그 트래픽 하한을 낮춘다.

주의 PIM(processing-in-memory)이 적당한 곳은 오직 write-heavy elementwise epilogue(약 1~3 MAC/elem)라는 소수 FLOP 지점뿐이다. 대부분의 FLOP(memory MLP forward/backward, NS-5)은 PIM이 못태우는 GEMM이라, “이 계열의 훈련이 새 메모리 소자를 요구한다”거나 “일반 PIM이 답이다”는 주장은 논문들 자신의 증거(알고리즘을 dense matmul로 빚어 기존 accelerator에 맞춤)

가 반대로 가리킨다. 이 계열이 memory-centric 논증을 펴는 정당한 지점은 **딱 둘** — **decode RMW 트래픽과 update-frequency ↔ tier** — 뿐이다.

9.11 정직성: 무엇을 믿고 무엇을이월하나

이 권의 모든 숫자는 하나의 계약 아래 읽어야 한다. 근거가 pre-silicon analytic cost model 과 CPU micro-benchmark 이기 때문이다.

- **load-bearing(믿어도 되는 것)**: 비율, crossover 위치, tier 순서, bound class. 세 방법 (달힌 형식 · analytic twin · hand-calc) 이 anchor 에서 1% 이내로 합치한 것들 — RMW 6.44 GB/token, state 134 MB/layer, read-crossover 65k, $394\times$ state-지배, $3.9\times$ cliff, C 곡선모양.
- **이월 (믿지 말 것)**: 절대 $\mu\text{s}/\text{token}\cdot\text{mJ}/\text{token}$ (1.92 ms, 46.5 mJ) 은 roofline 하한이라 사내 A100 runbook 으로 이월. novel scratchpad/PIM 이득 배율 ($6.8\times$, $1.6\times$) 은 directional DSE. $C^*\cdot\text{cliff}$ 의 절대 GB/s 는 host 좌표라 H100 으로 이전 안 됨 (모양만).

핵심 실전 판별법: “TTT decode 는 폭이 클수록 · 문맥이 65k 를 넘을수록 KV 보다 유리해진다” 는 load-bearing(비율 · crossover) 이다. “그 decode 가 1.92 ms 걸린다” 는 하한의 보고이지 주장이 아니다 — 그 자리엔 항상 “roofline 하한, A100 runbook 이월” 꼬리표가 붙는다.

요약 이 계열을 시스템으로 모델링하는 최소 골격은 여섯 문장이다. (1) 서빙상태는 KV cache 가 아니라 fast-weight $m d^2 L_{\text{layer}}$ 이고, 문맥이 아니라 폭으로 자란다. (2) decode 한 스텝은 매 토큰 그 state 전체를 read-modify-write 하므로 (6.44 GB/token, AI 0.59) **결정적으로 memory-bound** 이고 비용 = 트래픽이다 ($394\times$ 지배). (3) KV(append-once, 공유 가능) 와 TTT(대칭 RMW, unshared) 는 문맥 축에서 crossover($\sim 65k$) 로 영역을 나눈다. (4) batching 은 형성되되 (grouped-GEMM) 요청마다 weight 가 달라 상각이 없어 대역폭이 batch 를 조기에 닫는다 (B_{max} 벽). (5) 상태가 문맥 무관 고정이라 on-chip 상주가 설계 knob 이고, capacity 경계에서 대역폭이 $3.9\times$ 절벽으로 꺾이며, tier 는 update cadence 의 빠른 쪽이 정한다. (6) sizing 숫자가들 (params · 문맥) 에서 셋 (+state-bytes) 으로 늘고, capacity 는 retention 목표에서만 커지는 조건부 quality 축이다. 흥미로운 하드웨어 제안은 memory-centric 소자와 accelerator kernel 의 **쌍**이며, 모든절대치는 하한 · directional 로 이월된다.

이 권으로 booklet 세트의 시스템 관점 축이 닫힌다 — G00~G08 의 개념 (학습기초 · 빌딩블록 · Titans · Miras · Atlas · TNT · Nested Learning · Sleep) 을여기서 배운 state · 비용 · 계층 · scaling 의 언어로 다시 읽으면, 각 논문의 아키텍처 선택이 곧 하나의 시스템 비용 곡선으로 보일 것이다.